ALL SORTING ALGORITHMS

Interactive visualizers, step-by-step learning links and practice guidance

How to use this PDF

Open the link under the algorithm you are studying. Prefer Step / Next / Pause controls, enter a small custom array, and observe comparisons, swaps, partitions and merge operations. Some links are multi-algorithm visualizers: choose the named algorithm inside the website.

1. Basic comparison-based sorting

Bubble Sort

Repeatedly compares adjacent elements and swaps them when they are out of order.

• Interactive Bubble Sort visualizer

• CodeMonkey sorting visualizer

Selection Sort

Selects the smallest remaining element and places it at the next sorted position.

• USFCA sorting visualizations (select Selection Sort)

• CodeMonkey sorting visualizer

Insertion Sort

Builds a sorted prefix by inserting each next element into its correct position.

• USFCA sorting visualizations (select Insertion Sort)

• CodeMonkey sorting visualizer

Merge Sort

Divides the array into halves, sorts them recursively, then merges sorted halves.

• USFCA sorting visualizations (select Merge Sort)

• VisuAlgo sorting

Quick Sort

Partitions around a pivot so smaller values go to one side and larger values to the other.

• USFCA sorting visualizations (select Quick Sort)

• VisuAlgo sorting

Heap Sort

Builds a heap, repeatedly moves the maximum to the sorted end, and restores the heap.

• USFCA sorting visualizations (select Heap Sort)

• VisuAlgo sorting

Shell Sort

Performs insertion-sort-like passes over elements separated by decreasing gaps.

• USFCA sorting visualizations (select Shell Sort)

• Algorithm Visualizer

Cocktail Shaker Sort

A bidirectional Bubble Sort: one pass moves a large value right, the next moves a small value left.

• Algorithm Visualizer

• Sorting visualizer collection

Comb Sort

Uses a shrinking gap to compare distant elements, then finishes with adjacent comparisons.

• Algorithm Visualizer

• Sorting visualizer collection

Gnome Sort

Moves backward when adjacent values are out of order and forward when they are ordered.

• Algorithm Visualizer

2. Non-comparison and special-purpose sorting

Counting Sort

Counts how often each key occurs; especially useful when the integer range is small.

• VisuAlgo sorting

• Algorithm Visualizer

Radix Sort

Sorts values digit by digit (or by character position), often using a stable sub-sort.

• VisuAlgo sorting

• Algorithm Visualizer

Bucket Sort

Distributes values into buckets, sorts each bucket, then concatenates them.

• Algorithm Visualizer

• OpenDSA sorting modules

Pigeonhole Sort

Places values into indexed holes when the key range is small.

• OpenDSA sorting modules

3. One-stop visualizer websites

VisuAlgo — Sorting

Interactive sorting visualizations with controls to step through algorithm behaviour.

https://visualgo.net/en/sorting

USFCA — David Galles visualizations

Classic interactive visualizations for common sorting algorithms; choose the algorithm from the page.

https://www.cs.usfca.edu/~galles/visualization/

CodeMonkey — Sorting Algorithms

Friendly animated comparison of sorting methods.

https://www.codemonkey.com/interactive-cs-tools/sorting-algorithms/

Algorithm Visualizer

Explore algorithm animations and implementations; availability varies by algorithm.

https://algorithm-visualizer.org/

OpenDSA — Sorting

More detailed learning material and visualizations for sorting concepts.

https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/Sorting.html

Sorting.at

Visual sorting playground; select algorithms and inspect their behaviour.

https://sorting.at/

4. What to observe for every algorithm

• Comparisons: which values are being compared, and how often?

• Swaps / writes: when does an element move, and how far?

• Passes / partitions / merges: what part of the array becomes settled after each stage?

• Duplicates: test repeated values and check whether equal elements keep their relative order (stability).

• Edge cases: try an already sorted array, reverse-sorted array, all-equal values and a one-element array.

• Complexity: compare best, average and worst-case time, plus auxiliary space.

5. Suggested practice order

Bubble → Selection → Insertion → Merge → Quick → Heap → Counting → Radix → Bucket → Shell. Learn the first six thoroughly for DSA interviews; study the remaining algorithms for range-specific use cases and broader coverage.

Important: Website layouts and available algorithms can change. If a link opens a general visualizer page, select the algorithm named in its link description. The dedicated algorithm links are listed where available; multi-algorithm tools are clearly labelled.